Deliveroo: Real-Time DynamoDB Aggregations

Sharded Counters, Bucket-Tiled Sliding Windows & DynamoDB Streams CDC

STREAM AGGREGATIONS

LOGISTICS REAL-TIME

1. Real-Time Capacity Tracking at Deliveroo Scale

In on-demand food delivery networks, dispatch engines must evaluate real-time operational capacity across tens of thousands of restaurants and delivery zones every second:

Critical Real-Time Questions

- **Kitchen Saturation:** Has Restaurant R received > 25 orders in the trailing 15 minutes? If so, throttle storefront ordering to prevent kitchen bottlenecks.
- **Rider Density:** How many active couriers completed deliveries in Sector S in the last 10 minutes?
- **Surge Pricing:** Dynamically compute supply/demand imbalances at sub-5ms latencies.

2. The Hot-Partition Write Ceiling in DynamoDB

Amazon DynamoDB partitions tables by partition key (PK). However, an individual physical partition is strictly capped at **1,000 Write Capacity Units (WCU) per second**:

- During lunchtime dinner rushes, hundreds of orders and order status events hit trending restaurant chains within seconds.
- If all mutations increment a single counter record PK="`restaurant#101`", DynamoDB throttles requests with `ProvisionedThroughputExceededException`, causing order processing delays.

3. The Sharded Counter Architecture

Deliveroo eliminates partition write hotspots by splitting counter records into M independent sub-counters:

```
Write Path (Random Fan-Out):
1. Inbound Order for Restaurant R101 at 19:30 UTC.
2. Generator picks random shard S in [0, M-1] (e.g. M=10).
3. Executes DynamoDB atomic increment:
   UPDATE aggregates_table SET count = count + 1
   WHERE PK = "R101#2026-01-16T19:30" AND SK = "shard#S"
   -- Max throughput scales linearly: 10 * 1,000 = 10,000 WCU/sec!

Read Path (Parallel Scatter-Gather):
1. Client requests rolling count for R101 at 19:30.
2. Query fetches all M shards under PK="R101#2026-01-16T19:30" via single BatchGetItem (<4ms).
3. Client sums M values in memory: Total = Sum(shard_0 ... shard_M-1).
```

4. Bucket Tiling for Rolling Window Evaluation

Rather than running continuous range queries, Deliveroo slices continuous time into discrete **1-minute bucket tiles**:

```
Discrete 1-Minute Tiles: [T-14] [T-13] [T-12] ... [T-2] [T-1] [T_now]
                         ◄──────────────── 15-Minute Window ────────────────►

To calculate trailing 15-minute order volume:
1. Dispatch batch read for the exact 15 timestamp keys:
   Keys = ["R101#T_now", "R101#(T_now - 1m)", ..., "R101#(T_now - 14m)"]
2. DynamoDB returns pre-aggregated 1-minute summaries in a single round-trip.
3. Total Volume = Sum of 15 integer values.
Result: Sub-5ms evaluation without scanning full historical tables!
```

5. Event-Driven CDC Aggregation with DynamoDB Streams

Aggregates are kept in sync without burdening the transactional order-processing path:

DynamoDB Streams Log
Order state mutations (CREATED, CONFIRMED, CANCELLED) append to DynamoDB Streams in strict chronological order with zero database locking overhead.

Aggregator Stream Worker
AWS Lambda / Kinesis Analytics consumers batch events, compute net deltas (+1 for order, -1 for cancellation), and issue batched atomic increments to the aggregates table.

6. Aggregation Strategy Comparison Matrix

Architecture	Query Latency	Write Scalability	Data Freshness
SQL Full Table Scan	150ms – 1,500ms (High lock wait)	Severely limited (DB CPU bottleneck)	Immediate, but degrades system
Redis Sliding Window (ZSET)	Sub-3ms	Capped by single-thread Redis master	Near-instantaneous
Deliveroo Sharded DynamoDB	**Sub-5ms p99**	**10,000+ WCU / sec per key**	**Sub-second (Streams CDC)**

7. Zero-Cost Storage Pruning with DynamoDB TTL

Generating millions of 1-minute bucket records daily would quickly inflate table sizes and degrade index efficiency:

- **Automatic Lifecycle Expiration:** Every bucket record is written with an attribute `ttl_epoch = now() + 2 hours`.
- **Background Serverless Deletions:** DynamoDB's internal sweeper detects expired items and evicts them in the background.
- **Zero Consumed WCU:** Native TTL purges consume **0 Write Capacity Units** and incur zero billing cost, maintaining a constant table footprint indefinitely.

8. Key Architectural Takeaways for Real-Time Systems

Core Principles Distilled from Deliveroo's Aggregation Architecture

- **Shard Keys to Break Physical Partition Limits:** When write concurrency to an individual database record exceeds 1,000 WCU/sec, randomize mutations across M sub-shards and sum them on read.
- **Tile Continuous Time into Discrete Buckets:** Replace expensive sliding window range queries with discrete 1-minute bucket reads, summing the last K buckets in memory in sub-5ms.
- **Decouple Aggregation from Transaction Paths:** Use CDC streams (DynamoDB Streams / Debezium) to update counters asynchronously, keeping checkout latency completely isolated from analytical calculations.
- **Exploit Native TTLs for Maintenance-Free Hygiene:** Tag ephemeral temporal buckets with TTL timestamps to let the cloud platform clean up expired records without paying for deletion capacity.